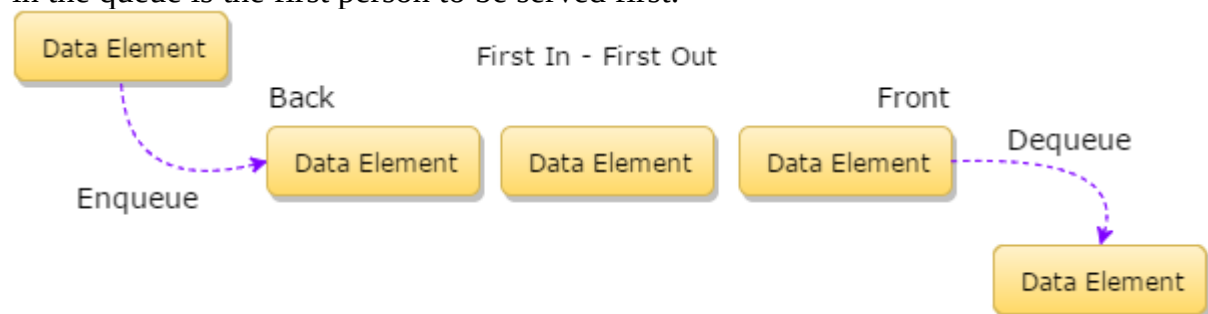


What is Queue?

A queue is a linear list of elements in which deletion of an element can take place only at one end called the front and insertion can take place on the other end which is termed as the rear. The term front and rear are frequently used while describing queues in a linked list. In this chapter, you will deal with the queue as arrays.

In the concept of a queue, the first element to be inserted in the queue will be the first element to be deleted or removed from the list. So Queue is said to follow the FIFO (First In First Out) structure. A real-life scenario in the form of example for queue will be the queue of people waiting to accomplish a particular task where the first person in the queue is the first person to be served first.



Other examples can also be noted within a computer system where the queue of tasks arranged in the list to perform for the line printer, for accessing the disk storage, or even in the time-sharing system for the use of CPU. So basically queue is used within a single program where there are multiple programs kept in the queue or one task may create other tasks which must have to be executed in turn by keeping them in the queue.

Queue as an ADT (Abstract Data Type)

The meaning of an abstract data type clearly says that for a data structure to be abstract, it should have the below-mentioned characteristics:

- First, there should be a particular way in which components are related to each other
- Second, a statement for the operation that can be performed on elements of abstract data type must have to be specified

Thus for defining a Queue as an abstract data type, these are the following criteria:

- Initialize a queue to be empty

- Check whether a queue is empty or not
- Check whether a queue is full or not
- Insert a new element after the last element in a queue, if the queue is not full
- Retrieve the first element of the queue, if it is not empty

Delete the first element in a queue, if it is not empty

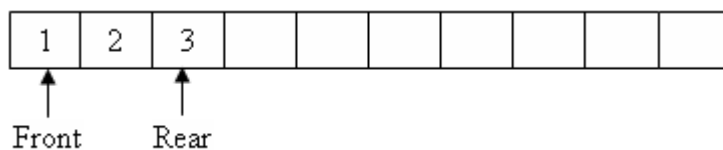
Definition and Concept

Queue is a sub class of linear data structure in which elements can be inserted from one end called rear and deleted from the other end called front. Since insertion and deletion in Queue is performed from two different ends, its elements are removed from queue in same order in which they were inserted. This means first item added in the list is the first item to be removed. Because of this characteristic queue is called FIFO (First In First Out) list.

Real life example of queue is people waiting in a line to purchase a movie ticket. The first person in line will get ticket first. Another example is payment counter in shopping mall. Time sharing system is also an example of queue.

According to data structure used for implementation queues can be static queue or dynamic queue. Queues implemented using fixed length array is called static queue and queues implemented using linked list is called dynamic queue. In this chapter only static queue is discussed. Dynamic queue is discussed with linked list.

According to the way the queues are implemented they can be separated into simple queue, circular queue and dequeue.



4.2 Operations On Simple Queue

Operations on queue include

Insert an element into queue.

Delete an element from queue.

4.2.1 Insertion In Simple Queue



Fig. 4.2 Empty Queue

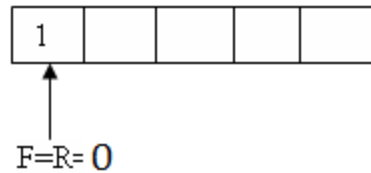


Fig. 4.3 Insertion In Empty Queue

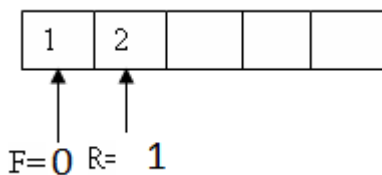


Fig. 4.4 Insertion In Non-Empty Queue

An algorithm to insert a value in simple queue.

QINSERT (Q, front, rear, size, Item)

Q: It is an array representing queue.

front: Indicates index for front element in array representing queue.

rear: Indicates index for rear element in array representing queue.

size: Maximum number of elements that can be stored in queue.

item: Value to be inserted in queue.

(Initially front=rear=-1)

Step 1: [check for overflow condition]

If rear=size then

Output "Queue Overflow"

Exit

Step 2:[When Queue is initially empty]

If front=NULL then

front=0

rear=0

Step-3: [Insert an Element]

Q[rear]=item

Step-4: [Increment Rear pointer]

rear=rear+1

An algorithm to delete a value from simple queue.

QINSERT (Q, front,rear, Item)

Q: It is an array representing queue.

front: Indicates index for front element in array representing queue.

rear: Indicates index for rear element in array representing queue.

item: Value to be deleted from the queue.

Step 1: [check for underflow condition]

 If front=NULL then

 Output "Queue Underflow"

 Exit

 Else

 [Remove an Element]

 Item= Q[front]

Step-2: [Check when the queue has only one element]

 If front==rear then

 front=NULL

 rear =NULL

 else

 front=front+1

Step-3:Exit

```

#include<iostream.h>
#include<conio.h>
#include<stdlib.h>

class queue
{
    int q1[5];
    int rear, front;

public:
    queue()
    {
        rear = -1;
        front = -1;
    }
    void insert(int x)
    {
        if (rear == 5) {
            cout << "queue over flow";
            rear=5;
            return;
        }
        if(front==-1)
        {
            front=0;
            rear=0;
        }
        q1[rear] = x;
        rear++;
    }

    void delet()
    {
        cout<<"\n value of front is \n"<<front<<":"<<rear;
        if (front == -1) {
            cout << "queue under flow";
            return;
        }
        else
        {
            cout << "deleted " << q1[front];
        }
        if(front==(rear-1))
        {
            front=-1;
            rear=-1;
        }
    }

```

```

    }
    else
    {
        front++;
    }

}

void display();
};
void queue::display()
{
    int i;
    if (front==-1)
    {
        cout << " queue empty";
        return;
    }
    for (i = front ; i < rear; i++)
        cout << q1[i] << " ";
}

int main()
{
    clrscr();
    int ch;
    queue qu;
    while (1)
    {
        cout << "\n1.insert 2.delet 3.display 4.exit\nenter ur choice: "; cin >> ch;
        switch (ch)
        {
            case 1:
                cout << "enter the element: "; cin >> ch;
                qu.insert(ch);
                break;
            case 2:
                qu.delet();
                break;
            case 3:
                qu.display();
                break;
            case 4:
                exit(0);
        }
    }
}

```

4.3 Circular Queue

In simple queue we can not utilize memory properly in some cases like if we have inserted 5 elements in queue with capacity of 5, then the queue is full. Now, we have deleted 2 elements from that queue then also we can not insert new element until whole queue gets empty, though it has 2 free elements. But in the case of circular queue the space of deleted elements can be used to insert new elements.

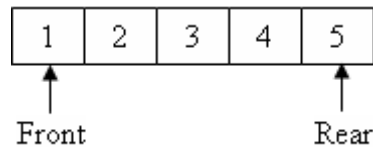


Fig. 4.8 Simple Queue With Overflow On Insert

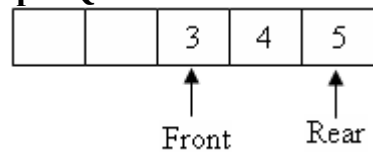
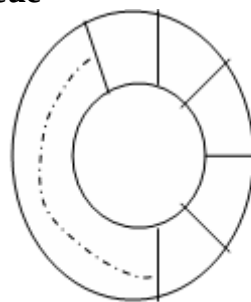


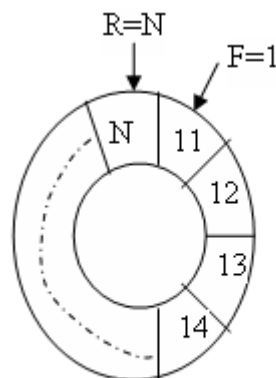
Fig. 4.9 Simple Queue With Overflow On Insert Though Having Space For 2 Elements

4.3.1 Insertion In Circular Queue



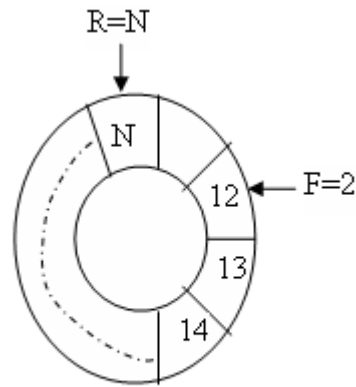
Initially Queue is empty.
 $F=R=0(\text{NULL})$

Fig. 4.10 Empty Queue



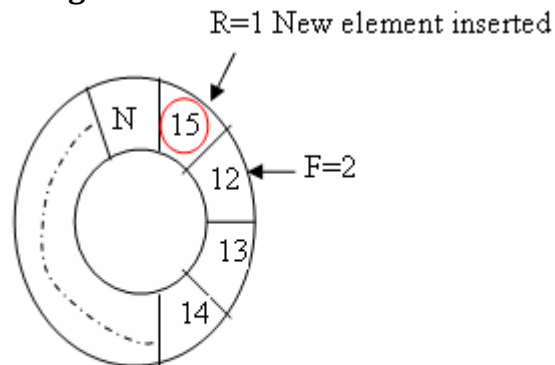
After insertion of N elements
 $F=1$ and $R=N$ Queue is full.

Fig. 4.11 1st sub part of queue overflow condition ($F=1$ and $R=N$)



After deletion of one element
F=2 and R=N

Fig. 4.12 Rear At the End



After insertion of one more element
F=2 and R=1
Again Queue is full

Fig. 4.13 2nd sub part of overflow condition ($F=R+1$)

An algorithm to insert element into circular queue.

CQINSERT (Q, R, N, F, Item)

Q: Array containing circular queue

F: Indicates index for front element in array representing circular queue.

R: Indicates index for rear element in array representing circular queue.

N: Maximum number of elements that can be stored in circular queue.

Item: Value to be inserted in circular queue.

[Initially F & R are set to zero]

Step1: [check for overflow]

(if $F=0$ and $R=N-1$) OR ($F=R+1$) then

Write "queue overflow on insert"

Exit

Step2:[Update F and R pointer according to the need]


```

[check for insertion of first element in circular queue]
If F=-1 then
    F:=0
    R:=0
    [check if rear at the end of circular Queue]
Else if R=N-1 then
    R:=0
Else
    R:=R+1
[End if structure]
Step 3: [Insert the element]
    Set Q[R]:=item
Step 4: Exit

```

Algorithm for deletion from circular Queue

```

Step 1: [Check Underflow]
    If F=-1 then
        Write "queue underflow"
        Exit
    [End if structure]
Step 2: [Get the element to be deleted]
    Item:=Q[F];
Step 3:[Reset F & R]
    [check for deletion of last element from Queue]
    If F=R then
        F:=-1
        R:=-1
        [check if front at the end of circular Queue]
    Else
        if F=N then
            F:=0
        Else
            F:=F+1
    [End if structure]

```

```

#include<iostream.h>
#include<conio.h>
#include<stdlib.h>
#define size 5
//program for circular queue
class cir
{
    int Q[size];
    int rear, front;

public:
    cir()
    {
        rear = -1;
        front = -1;
    }
    void insert()
    { int no;
      if((front==0 && rear==size-1)|| (front==rear+1))
      {
          cout<<"\n overflow";
          return;
      }
      else if(front==-1)
      {
          front=0;
          rear=0;
      }
      else if(rear==size-1)
      {
          rear=0;
      }
      else
      rear++;
      cout<<"\n Enter element :";
      cin>>no;
      Q[rear]=no;

    }
    void del()
    {
        if(front==-1)
        {
            cout<<"\n underflow ";
            return;
        }
    }

```

```

    cout<<"\n Element deleted:"<<Q[front];
    if(front==rear)
    {
        front=-1;
        rear=-1;
    }
    else
    {
        if(front==size-1)
            front=0;
        else
            front++;
    }
}

void print()
{
    int i;
    if(front==-1)
    {
        cout<<"\n queue is empty ";
        return;
    }
    if(front<=rear)
    {
        for(i=front;i<=rear;i++)
            cout<<"\n"<<Q[i];
    }
    else
    {
        for(i=front;i<size;i++)
            cout<<"\n"<<Q[i];
        for(i=0;i<=rear;i++)
            cout<<"\n"<<Q[i];
    }
}

};

int main()
{
    int ch;
    cir qu;
    while (1) {
        cout << "\n1.insert 2.delet 3.display 4.exit\nenter ur choice: "; cin >> ch;

```

```

switch (ch) {
case 1:
    qu.insert();
    break;
case 2:
    qu.del();
    break;
case 3:
    qu.print();
    break;
case 4:
    exit(0);
}
}
}

```

Double Ended Queue (Deque)

Double ended queue is a queue in which insertions and deletions are made to or from either or both ends of the queue. There are two variations of double ended queue, **input restricted deque** and **output restricted deque**. The restricted operation can only be performed from one end. In input restricted deque insertion can be performed from only one end. In output restricted deque delete operation can be performed from only one end. In algorithms deque using circular queue is considered.

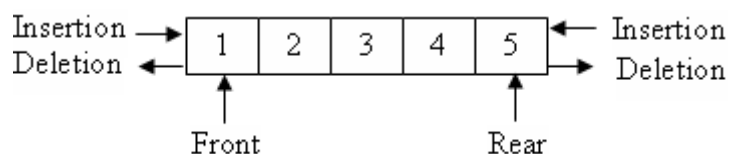


Fig. 4.17 Deque Using Simple Queue

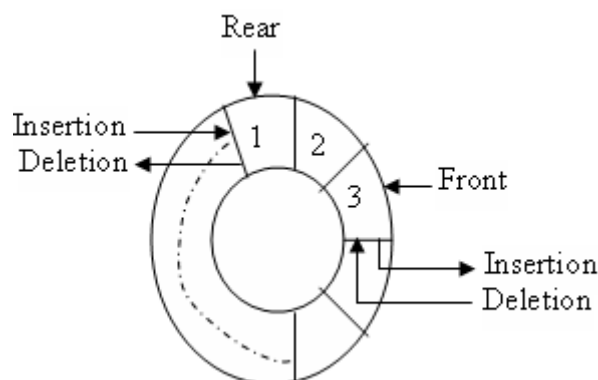


Fig. 4.18 Deque Using Circular Queue

Insertion In Double Ended Queue

An algorithm to insert an element into double ended queue.

DQ: Array containing a double ended queue.

DIR: Define which side you want to insert either Left or Right

Step 1: [Get the value of DIR]

Read DIR

Step 2: [Check for overflow condition]

If Left = 1 & Right = N or Left = Right+1 then

Write "Overflow on insertion"

Exit

[End of if structure]

Step 3: [Check if double ended queue is empty or not?]

If Left = Null then

Left: = 1

Right: = 1

DQ [Left]:= Item

Exit

[End of if structure]

Step 4: [Check if DIR is Left?]

If DIR = "Left" then

If Left = 1 then

Left: = N

Else

Left: = Left-1

[End of if structure]

DQ [Left]:= Item

Step 5: [Check DIR is Right]

Else If DIR = Right then

If Right = N then

Right: = 1

Else

Right: = Right+1

[End if structure]

DQ [Right]:= Item

[End of if structure]

Step 6: [Finished]

Exit

Deletion From Double Ended Queue

An algorithm to delete an element from double ended queue.

DQ: Array containing a double ended queue.

DIR: Define the side you want to delete either Left or Right. It indicates the direction of deletion.

Item: Contains the data for element being deleted.

Step 1: [Check for underflow]

 If Left = Null

 Output “Underflow on deletion”

 Exit

 [End if structure]

Step 2: [if deleting the last element]

 If Left = Right then

 Item: = DQ [Left]

 Left: = Null

 Right: = Null

 Return (Item)

 [End if structure]

Step 3: [Get the value of direction]

 Read DIR

Step 4: [Check if DIR is Left]

 If DIR = “Left” then

 Value: = DQ [Left]

 If Left = N then

 Left: = 1

 Else

 Left: = Left+1

 [End if structure]

Step 5: [Check if DIR is Right]

 Else If DIR = Right then

 Value: = DQ [Right]

 If Right = 1 then

 Right: = N

 Else

 Right: = Right-1

 [End if structure]

 [End if structure Step 4]

Step 6: Return (value)

```

//Write a program for Dequeue
#include<iostream.h>
#include<stdlib.h>
#include<conio.h>
#define max 3
class deq
{
private:
int deque_arr[max];
int front;
int rear;
public:
deq()
{
front=-1;
rear=-1;
}
void insert_frontend(int item);
void insert_rearend(int item);
int delete_frontend();
int delete_rearend();
void display();
};
void deq::insert_frontend(int item)
{

    if (((front==0) && (rear==max-1)) || (front==rear+1) )
    {
        cout<<"\nqueue overflow\n";
        return;
    }
    else if( front==-1 )/*if queue is initially empty*/
    {
        front=0;
        rear=0;
    }
    else if(front==0)
        front=max-1;
    else
        front=front-1;
}

```

```

        deque_arr[front]=item ;
    }/*end of insert_frontend()*/

void deq::insert_rearend(int item)
{
    if (((front==0) && (rear==max-1)) || (front==rear+1) )
    {
        cout<<"\nqueue overflow\n";
        return;
    }
    else if(rear==-1) /*if queue is initially empty*/
    {
        front=0;
        rear=0;
    }
    else if(rear==max-1) /*rear is at last position of queue */
        rear=0;
    else
        rear=rear+1;
    deque_arr[rear]=item ;
}/*end of insert_rearend()*/

```

```

int deq::delete_frontend()
{
    int item;
    if( front == -1)
    {
        cout<<"\nqueue underflow\n";
        exit(1);
    }

    item=deque_arr[front];
    if(front==rear) /*queue has only one element */
    {
        front=-1;
        rear=-1;
    }
    else
        if(front==max-1)
            front=0;

```



```

        else
            front=front+1;
    return item;
}/*end of delete_frontend()*/

int deq::delete_rearend()
{
    int item;
    if( front == -1)
    {
        cout<<"\nqueue underflow\n";
        exit(1);
    }
    item=deque_arr[rear];

    if(front==rear) /*queue has only one element*/
    {
        front=-1;
        rear=-1;
    }
    else if(rear==0)
        rear=max-1;
    else
        rear=rear-1;
    return item;
}/*end of delete_rearend() */

```

```

void deq::display()
{
    int i;

    if( front == -1)
    {
        cout<<"\nqueue underflow\n";
        //exit(1);
        return;
    }
    cout<<"\nqueue elements :\n";
    i=front;

```

```

        if( front<=rear )
        {
            while(i<=rear)
                cout<<" "<<deque_arr[i++];
        }
        else
        {
            while(i<=max-1)
                cout<<" "<<deque_arr[i++];
            i=0;
            while(i<=rear)
                cout<<" "<<deque_arr[i++];
        }
        cout<<"\n";
    }/*end of display() */

/* c program to implement deque using circular array */
int main()
{
    deq q1;
    int choice,item;
    while(1)
    {
        cout<<"\n\n1.insert at the front end\n";
        cout<<"2.insert at the rear end\n";
        cout<<"3.delete from front end\n";
        cout<<"4.delete from rear end\n";
        cout<<"5.display\n";
        cout<<"6.quit\n";
        cout<<"\nenter your choice : ";
        cin>>choice;

        switch(choice)
        {
            case 1:
                cout<<"\ninput the element for adding in queue : ";
                cin>>item;
                q1.insert_frontend(item);
                break;
            case 2:

```

```

        cout<<"\ninput the element for adding in queue : ";
        cin>>item;
        q1.insert_rearend(item);
        break;
    case 3:
        cout<<"\nelement deleted from front end is
:\n"<<q1.delete_frontend();
        break;
    case 4:
        cout<<"\nelement deleted from rear end is :
\n"<<q1.delete_rearend();
        break;
    case 5:
        q1.display();
        break;
    case 6:
        exit(1);
    default:
        cout<<"\nwrong choice\n";
    }/*end of switch*/

    q1.display();
    }/*end of while*/
}/*end of main()*/

```

Applications of Circular Queue

- CPU scheduling
- Memory management
- Traffic Management